

DEEP LEARNING & COMPUTER VISION

Comprehensive Exam-Focused Study Notes

Topics Covered:

- Mathematics of Deep Learning
- Computer Vision Introduction
- Neural Network Fundamentals
- Backpropagation & Optimizers
- Convolutional Neural Networks (CNNs)
- CNN Architectures (AlexNet, VGG, Inception, ResNet)
 - Normalization & Regularization
- Recurrent Neural Networks (RNNs)
 - LSTM, GRU & Sequence Models
- Autoencoders & Segmentation
 - PyTorch Code Patterns

ePGD – Deep Learning & Generative AI | IIT Bombay + Great Learning
Prepared for Exam Revision | April 2026

TABLE OF CONTENTS

TABLE OF CONTENTS.....	2
------------------------	---

TOPIC 01: MATHEMATICS OF DEEP LEARNING

1.1 Intuition – Why Does Math Matter in Deep Learning?

Deep Learning is fundamentally a mathematical optimization problem. A neural network learns by adjusting millions of parameters to minimize a loss function. To do this, we need linear algebra (to represent data and transformations), calculus (to compute how parameters affect the loss), and probability theory (to model uncertainty).

1.2 Key Mathematical Concepts

Vectors and Matrices

A vector $x \in \mathbb{R}^n$ represents an input (e.g., pixel values). A matrix $W \in \mathbb{R}^{(m \times n)}$ represents a linear transformation (weights of a layer).

- Dot product: $x \cdot w = x_1 w_1 + x_2 w_2 + \dots + x_n w_n$ (measures similarity/alignment)
- Matrix multiplication: $y = Wx$ (linear layer computation)
- Transpose: $(AB)^T = B^T A^T$

Chain Rule (Foundation of Backpropagation)

If $y = f(g(x))$, then $dy/dx = (df/dg)(dg/dx)$. This is applied repeatedly in neural networks.

$$\partial L / \partial W = (\partial L / \partial a) \times (\partial a / \partial z) \times (\partial z / \partial W)$$

Where: L = loss, a = activation output, z = pre-activation, W = weight matrix

1.3 Activation Functions

Function	Formula & Notes
Sigmoid	$\sigma(x) = 1/(1+e^{-x})$ Range: (0,1) Suffers vanishing gradient
Tanh	$\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$ Range: (-1,1) Zero-centered
ReLU	$f(x) = \max(0, x)$ Derivative: 1 if $x > 0$ else 0 Dying ReLU issue
Leaky ReLU	$f(x) = \max(0.01x, x)$ Fixes dying ReLU
Swish	$f(x) = x \cdot \sigma(x)$ Smooth, non-monotonic, used in EfficientNet
Softmax	$\sigma(y_i) = e^{y_i} / \sum_j e^{y_j}$ Converts logits to probabilities (sum=1)

1.4 Loss Functions

Mean Squared Error (MSE) – for regression

$$L = (1/n) \sum (y_i - \hat{y}_i)^2$$

Cross-Entropy Loss – for classification

$$L = -(1/n) \sum [y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)]$$

For multi-class: $L = -\sum y_i \log(\hat{y}_i)$

Binary Cross-Entropy – for binary/reconstruction

$$L_{BCE} = -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$$

1.5 Gradient Descent Variants

Optimizer	Description
Batch GD	Uses entire dataset for each update. Stable but slow.
Mini-Batch GD	Uses small subset (batch). Best of both worlds. Most common.
SGD + Momentum	$w_{\text{new}} = w - \text{lr} \cdot \text{grad} + \text{momentum} \cdot v$ Helps escape local minima
Adam	Adaptive learning rate with momentum. $\text{lr}_{\text{eff}} = \alpha / (\sqrt{\hat{v}} + \epsilon)$. Most popular.
RMSProp	Divides lr by running average of squared gradients.

✗ COMMON MISTAKE: Forgetting that a high learning rate can cause divergence, while a very low learning rate makes training extremely slow.
 Typical starting point: lr = 0.001 for Adam, lr = 0.01 for SGD.

1.6 Sample Questions

[Easy] What does the softmax function do? Why is it used in the output layer?

Answer: Softmax converts raw output scores (logits) into probabilities that sum to 1. Formula: $\sigma(y_i) = e^{y_i} / \sum e^{y_j}$. It's used in multi-class classification because each output neuron then represents the probability of the input belonging to class i.

[Medium] Compute the gradient of sigmoid $\sigma(x)$. Why does this cause the vanishing gradient problem?

Answer: Derivative: $\sigma'(x) = \sigma(x) \cdot (1 - \sigma(x))$. Maximum value is 0.25 (at $x=0$). When stacked across many layers, multiplying 0.25^L (L layers) gives extremely small gradients. For 10 layers: $0.25^{10} \approx 9.5 \times 10^{-7}$ — almost zero, so early layers stop learning.

[Hard] Explain Adam optimizer. Why is it preferred over SGD with momentum?

Answer: Adam maintains: (1) 1st moment ($m = \beta_1 m + (1 - \beta_1)g$) — exponential moving average of gradients, similar to momentum; (2) 2nd moment ($v = \beta_2 v + (1 - \beta_2)g^2$) — exponential moving average of squared gradients for adaptive LR. Update: $w := w - \alpha \cdot \hat{m} / \sqrt{\hat{v} + \epsilon}$. Bias-corrected: $\hat{m} = m / (1 - \beta_1^t)$, $\hat{v} = v / (1 - \beta_2^t)$. Benefits: adapts LR per parameter, robust to sparse gradients, fast convergence. Typical: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 1e-8$.

TOPIC 02: INTRODUCTION TO COMPUTER VISION

2.1 What is Computer Vision?

Computer Vision is the field of AI that enables machines to interpret visual information (images, video). It sits at the intersection of image processing, machine learning, and geometry.

Field	Transformation
Image Processing	Image → Image (e.g., blurring, edge detection)
Computer Graphics	Knowledge/3D model → Image (rendering)
Computer Vision	Image → Knowledge (understanding what's in the image)
Deep Learning	Learned features automatically from data — powers modern CV

2.2 Core Vision Tasks

- Classification: Assign a single label to the entire image (e.g., 'cat' or 'dog')
- Object Detection: Locate AND classify multiple objects with bounding boxes
- Semantic Segmentation: Label every pixel with its class (road, sky, person...)
- Instance Segmentation: Detect individual object instances separately
- Pose Estimation: Estimate human body keypoints (joints, skeleton)
- Image Captioning: Generate text descriptions of images (Vision + NLP)

2.3 Challenges in Computer Vision

Challenge	Why It's Hard
Viewpoint Variation	Same object looks different from different angles
Illumination Changes	Lighting dramatically changes pixel values
Occlusion	Objects partially hidden behind other objects
Scale Variation	Same object can appear very large or very small
Intra-class Variation	Cats can look very different from each other
Background Clutter	Object blends into complex backgrounds

2.4 Historical Milestones

1. 1957: Perceptron (Rosenblatt) — First trainable model
2. 1969: Minsky & Papert — Showed perceptron can't solve XOR
3. 1980: Neocognitron (Fukushima) — Inspired by visual cortex, ancestor of CNNs
4. 1986: Backpropagation (Rumelhart, Hinton) — Training multilayer networks
5. 1998: LeNet (LeCun) — CNN for handwritten digit recognition
6. 2012: AlexNet (Krizhevsky) — 16.4% ImageNet error; Deep Learning era begins
7. 2014: VGG — 7.3% error; showed depth matters
8. 2015: ResNet (He) — 3.6% error; better than humans (5.1%)

2.5 Key Datasets

Dataset	Details
---------	---------

ImageNet-1K (ILSVRC)	1.28M training images, 1,000 classes. Standard benchmark.
ImageNet-21K	14.2M images, 21,841 classes. Pre-training dataset.
LAION-400M	400M image-text pairs. Used for CLIP training.
MS-COCO	High samples per class; benchmark for detection & segmentation.
PASCAL VOC	20 classes; used for segmentation evaluation (IoU metric)

2.6 Sample Questions

[Easy] What is the difference between semantic segmentation and instance segmentation?

Answer: Semantic segmentation assigns a class label to every pixel, but does not distinguish between different objects of the same class (all dogs get the same color). Instance segmentation detects individual object instances — each separate dog gets a unique mask/color. Example: 3 dogs → semantic gives 1 'dog' region; instance gives 3 separate dog masks.

[Medium] Why did AlexNet (2012) mark such a big breakthrough in computer vision?

Answer: AlexNet reduced ImageNet error from 26% (2011) to 16.4% — a 10% absolute improvement, unprecedented at the time. Key innovations: (1) Used ReLU instead of sigmoid — much faster training; (2) Trained on GPUs for efficiency; (3) Used Dropout for regularization; (4) Data augmentation. It proved deep CNNs trained end-to-end outperform hand-engineered features.

[Hard] What is the difference between image processing and computer vision? Give an example of each.

Answer: Image Processing: Input is an image, output is also an image (transformation). No semantic understanding. Example: Applying a Gaussian blur to reduce noise, or Sobel filter to detect edges — these operate purely on pixel values. Computer Vision: Input is an image, output is knowledge/understanding. Involves semantic interpretation. Example: Detecting that an image contains a 'pedestrian' at coordinates (x,y,w,h) — the system understands what it sees, not just transforms pixels.

TOPIC 03: NEURAL NETWORK FUNDAMENTALS

3.1 The Perceptron

A perceptron is the simplest neural unit. It computes a weighted sum of inputs and applies a step function.

$$z = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b = \mathbf{w}^T \mathbf{x} + b$$

$$\text{output} = 1 \text{ if } z \geq 0, \text{ else } 0$$

Where: x = input features, w = weight vector, b = bias, z = pre-activation value

- Intuition: Think of a perceptron as a voter. Each input x_i casts a 'vote' weighted by w_i . The bias b sets the threshold. If the weighted vote total exceeds the threshold, output = 1.

3.2 XOR Problem — Why We Need Hidden Layers

The XOR function cannot be solved by a single perceptron because XOR is NOT linearly separable — no single straight line can separate the classes. This was proven by Minsky & Papert (1969).

XOR truth table: (0,0)→0, (0,1)→1, (1,0)→1, (1,1)→0

Solution: Add hidden layers. Each hidden layer creates a new feature space where the problem becomes linearly separable.

With 1 hidden layer of 2 units: First learn NAND and OR, then AND them.

3.3 Multi-Layer Perceptron (MLP) Architecture

An MLP has: Input Layer → Hidden Layer(s) → Output Layer. Each hidden layer applies a linear transformation followed by a non-linear activation.

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$

$$a^{(l)} = f(z^{(l)})$$

Where: l = layer index, $W^{(l)}$ = weight matrix, $b^{(l)}$ = bias, f = activation function, a = activations

3.4 Universal Approximation Theorem

THEOREM: A feedforward neural network with ONE hidden layer containing SUFFICIENT neurons and a NON-LINEAR activation function can approximate ANY continuous function on a compact (closed, bounded) set to ARBITRARY accuracy.

What it means: In theory, a 2-layer network can learn anything.

What it does NOT say: It doesn't tell you how many neurons are needed (could be exponential), or that training will find the right parameters, or that it will generalize to new data.

Practical implication: Deep networks are more EFFICIENT than wide networks for complex tasks. Depth creates $O(m^L)$ linear regions; width creates only $O(m)$ — exponential advantage for depth!

3.5 Depth vs Width

Property	Wide Network (More Neurons) vs Deep Network (More Layers)
----------	---

Parameters	Wide: increases linearly Deep: can be exponentially more efficient
Linear Regions	Wide with m units: $O(m)$ regions Deep with m units per layer, L layers: $O(m^L)$
Learning Style	Wide: tends to memorize Deep: learns hierarchical representations
Training	Wide: faster to train Deep: harder, needs BatchNorm, skip connections
Generalization	Wide: higher overfitting risk Deep: better generalization with proper regularization

3.6 Sample Questions

[Easy] What is the role of the bias term in a neural network?

Answer: The bias term b shifts the activation threshold independently of the input. Without bias, every hyperplane defined by the neuron must pass through the origin, severely limiting what functions the network can learn. With bias, the decision boundary can be positioned anywhere in the input space. In code: `nn.Linear(in, out, bias=True)` includes bias by default.

[Medium] Why do we need non-linear activation functions? What happens if all activations are linear?

Answer: If all activations are linear, the entire network collapses to a single linear transformation regardless of depth: $W_3 (W_2 (W_1 x)) = Wx$. The network cannot learn any non-linear function (like XOR, or recognizing curves in images). Non-linear activations break this by introducing 'kinks' (ReLU) or 'squashing' (sigmoid) — each layer creates a new non-linear feature representation, enabling learning of complex patterns.

[Hard] Why do deep networks generalize better than wide networks for many practical tasks, given that both can approximate any function?

Answer: While both can approximate any function (Universal Approximation), deep networks are more parameter-efficient: D hidden layers with m neurons each create $O(m^D)$ linear regions vs $O(m)$ for a single layer. This means deep networks can represent complex functions using fewer parameters, which reduces the risk of overfitting. Additionally, deep networks learn hierarchical representations — e.g., pixels→edges→shapes→objects — that match the hierarchical structure of real-world data. The inductive bias of depth aligns with how visual information is structured. Generalization gap scales as $O(\sqrt{\text{capacity}/n})$, so fewer parameters with same expressivity = better generalization.

TOPIC 04: BACKPROPAGATION & TRAINING

4.1 The Training Loop

9. Forward Pass: Feed input through network, compute prediction $\hat{y}$
10. Compute Loss: $L = \text{loss_function}(\hat{y}, y_{\text{true}})$
11. Backward Pass: Compute gradients $\partial L / \partial W$ for each weight W
12. Update Weights: $W = W - \text{lr} \times \partial L / \partial W$
13. Repeat until convergence

4.2 Backpropagation Algorithm

Backprop is just the chain rule applied systematically through the computation graph, going backwards from the loss.

$$\begin{aligned} \partial L / \partial W^{(l)} &= \partial L / \partial a^{(l)} \times \partial a^{(l)} / \partial z^{(l)} \times \partial z^{(l)} / \partial W^{(l)} \\ \delta^{(l)} &= (W^{(l+1)})^T \delta^{(l+1)} \odot f'(z^{(l)}) \quad [\text{error at layer } l] \end{aligned}$$

Where: $\delta^{(l)}$ = error/delta at layer l , $\odot$ = element-wise multiplication, f' = derivative of activation

4.3 Vanishing and Exploding Gradients

VANISHING GRADIENT: Gradients shrink to near-zero as they propagate backward.
 Cause: Multiplying many small numbers (<1) together. e.g., $0.25 \times 0.25 \times 0.25 = 0.016$
 Effect: Early layers barely learn; network effectively becomes shallow.
 Symptom: Loss barely decreases, weights in early layers don't change.

EXPLODING GRADIENT: Gradients grow exponentially.
 Cause: Multiplying many large numbers (>1) together. e.g., $2.5^3 = 15.625$
 Effect: Weights blow up to NaN; training becomes unstable.
 Symptom: Loss returns NaN, weights become extremely large.

LAYER 1 GRADIENT FORMULA:
 $\partial L / \partial W^{(1)} = \delta^{(L)} \times W^{(L)} \times W^{(L-1)} \times \dots \times W^{(2)} \times x$

4.4 Solutions to Gradient Problems

Problem	Solution
Vanishing gradient	Use ReLU (gradient=1 when $x>0$), ResNet skip connections, BatchNorm
Exploding gradient	Gradient clipping (clip norm to max value), weight initialization
General instability	Batch Normalization normalizes activations per mini-batch
Deep training issues	Xavier/He initialization, Learning rate scheduling

4.5 PyTorch Training Code Pattern

```
# Standard PyTorch training loop:
for epoch in range(num_epochs):
```

```

for X_batch, y_batch in dataloader:
    optimizer.zero_grad()    # Clear gradients from previous step
    pred = model(X_batch)    # Forward pass
    loss = criterion(pred, y_batch) # Compute loss
    loss.backward()          # Compute gradients (backprop)
    optimizer.step()         # Update weights

```

CRITICAL: Never forget `zero_grad()` — PyTorch accumulates gradients by default!

4.6 Sample Questions

[Easy] What does `optimizer.zero_grad()` do and why is it important?

Answer: PyTorch accumulates (adds) gradients in the `.grad` attribute of each parameter by default. `zero_grad()` sets all gradients to zero before the next backward pass. Without it, gradients from multiple batches add up incorrectly, making the weight update wrong. Always call `zero_grad()` at the start of each training step before `loss.backward()`.

[Medium] Trace through one training step: $x=2.0$, $w=1.0$, $b=0.5$, $\text{target}=3.0$, $\text{loss}=(\text{pred}-\text{target})^2$, $\text{lr}=0.5$. What is the new weight after one step?

Answer: Step 1 - Forward: $\text{pred} = wx + b = 1.0 \times 2.0 + 0.5 = 2.5$ Step 2 - Loss: $L = (2.5 - 3.0)^2 = (-0.5)^2 = 0.25$ Step 3 - Gradients: $\partial L / \partial \text{pred} = 2(\text{pred} - \text{target}) = 2 \times (-0.5) = -1.0$ $\partial L / \partial w = \partial L / \partial \text{pred} \times \partial \text{pred} / \partial w = -1.0 \times x = -1.0 \times 2.0 = -2.0$ $\partial L / \partial b = \partial L / \partial \text{pred} \times \partial \text{pred} / \partial b = -1.0 \times 1 = -1.0$ Step 4 - Update: $w_{\text{new}} = 1.0 - 0.5 \times (-2.0) = 1.0 + 1.0 = 2.0$; $b_{\text{new}} = 0.5 - 0.5 \times (-1.0) = 1.0$

[Hard] In PyTorch, what is `y.grad` for: $x=3.0$, $y=4.0$, $z = x^2 + 2xy + y^3$?

Answer: Compute $\partial z / \partial y = 2x + 3y^2 = 2(3) + 3(4^2) = 6 + 48 = 54$. So `y.grad` = 54.0. Note: $z = 9 + 24 + 64 = 97$; $\partial z / \partial x = 2x + 2y = 6 + 8 = 14$, so `x.grad` = 14.0.

TOPIC 05: CONVOLUTIONAL NEURAL NETWORKS (CNNs)

5.1 Why CNNs over MLPs for Images?

Issue with MLP	How CNN Solves It
Loses spatial structure (flattening)	Processes 2D grid — preserves H×W×C structure
Too many parameters (256×256 image = 65K+ nodes)	Parameter sharing: same filter across entire image
Not translation invariant	Same filter detects features anywhere in image
Treats all pixels equally (no locality)	Local receptive field — adjacent pixels matter more

5.2 Convolution Operation

A filter (kernel) of size $k \times k$ slides across the image. At each position, element-wise multiplication is performed, and results are summed to produce ONE output pixel (feature map value).

$$\text{Output}[i,j] = \sum_m \sum_n \text{Input}[i \cdot s + m, j \cdot s + n] \times \text{Filter}[m,n] + \text{bias}$$

Where: s = stride, m, n iterate over kernel dimensions

- Visual Intuition: Imagine placing a 3×3 magnifying lens over the image. The lens (filter) detects ONE type of feature (e.g., horizontal edge). Slide the lens across every position → you get a 'feature map' showing where that feature appears. Use 64 different filters → 64 feature maps, each detecting a different pattern.

5.3 Output Size Formula — CRITICAL!

$$\text{Output_size} = \text{floor}((\text{Input_size} - \text{Kernel_size} + 2 \times \text{Padding}) / \text{Stride}) + 1$$

Always memorize this formula! Exam questions frequently ask for dimension calculations.

Parameter	Effect on Output
Larger Kernel	More context, fewer output pixels
Stride > 1	Downsampling — output gets smaller
'Same' Padding	Output size = Input size (zero-pads edges, stride=1)
'Valid' Padding	No padding, output shrinks: Output = Input - Kernel + 1
Dilation rate r	Effective kernel size = $(k-1) \times r + 1$ — expands receptive field without more params

Example: Input 32×32 , kernel 5×5 , stride 2, padding 1:

$$\text{Output} = \text{floor}((32 - 5 + 2 \times 1) / 2) + 1 = \text{floor}(29/2) + 1 = 14 + 1 = 15$$

5.4 Multi-Channel Convolution

For RGB images: filter has same depth as input (3 channels). Filter shape: $k \times k \times C_{\text{in}}$. ALL channels are processed, then summed to produce ONE output feature map value.

$$\text{Output} = \sum_{c=1}^{C_{\text{in}}} (\text{Filter}_c * \text{Input}_c) + \text{bias}$$

If we use C_{out} filters total: Output shape = $H_{\text{out}} \times W_{\text{out}} \times C_{\text{out}}$

5.5 Pooling Layers

Pooling Type	Operation & Purpose
Max Pooling	Takes maximum value in each patch. Extracts strongest feature. Most common.
Average Pooling	Computes average. Smoother, used in final classification layers.
Global Avg Pool (GAP)	Single avg value per feature map. Replaces flatten+FC. Reduces params.
No Learned Params	Pooling has no weights to train! Only hyperparams: window size, stride.

5.6 Dilated (Atrous) Convolution

Dilated convolution inserts spaces between kernel elements, effectively increasing the receptive field without increasing parameters or losing resolution.

$$\text{Effective Kernel Size} = (\text{kernel_size} - 1) \times \text{dilation_rate} + 1$$

Example: 3×3 kernel with dilation=2: effective size = (3-1)×2+1 = 5×5 (but only 9 actual parameters!)

KEY INSIGHT: Dilated convolution is critical in segmentation models (DeepLab) to maintain spatial resolution while increasing receptive field. Stride reduces resolution — dilation doesn't! ASPP (Atrous Spatial Pyramid Pooling) uses multiple dilation rates (6, 12, 18) simultaneously.

5.7 Parameter Count

For a Conv2d layer: $\text{params} = \text{kernel_h} \times \text{kernel_w} \times C_{\text{in}} \times C_{\text{out}} + C_{\text{out}}$ (bias)

Example: Conv2d(3→16, k=5): $5 \times 5 \times 3 \times 16 + 16 = 1200 + 16 = 1216$ parameters

For Linear layer: $\text{params} = \text{in_features} \times \text{out_features} + \text{out_features}$ (bias)

5.8 Sample Questions

[Easy] What is the purpose of padding='same' in a convolutional layer?

Answer: 'Same' padding adds zeros around the input borders so that the output has the same height and width as the input (when stride=1). This prevents the feature map from shrinking with each layer, which would limit network depth. It also ensures edge pixels get equal treatment — without padding, corner pixels only contribute to one convolution operation, but center pixels contribute to many.

[Medium] CNN with Input (4,3,32,32): Conv2d(3,16,k=5,s=2,p=1), Conv2d(16,32,k=3,s=1,p=0), MaxPool2d(k=2,s=2). What is the output shape?

Answer: After Conv1: Output = $\text{floor}((32+2-5)/2)+1 = \text{floor}(29/2)+1 = 15$. Shape: (4,16,15,15) After Conv2: Output = $\text{floor}((15-3)/1)+1 = 13$. Shape: (4,32,13,13) After MaxPool: Output = $\text{floor}((13-2)/2)+1 = 6$. Shape: (4,32,6,6) Total feature map: $32 \times 6 \times 6 = 1152$ features

[Hard] What is dilated convolution? How does a 3×3 kernel with dilation=6 relate to its receptive field?

Answer: Dilated (atrous) convolution inserts (dilation-1) zeros between kernel weights. A 3×3 kernel with dilation=6 has effective size: $(3-1) \times 6 + 1 = 13 \times 13$, covering a 13×13 region of the input but with only 9

learnable parameters (same as a regular 3×3). This is crucial in DeepLab's ASPP: dilation rates of 6,12,18 give effective kernels of 13×13 , 25×25 , 37×37 simultaneously — capturing multi-scale context without losing resolution.

TOPIC 06: CNN ARCHITECTURES: ALEXNET, VGG, INCEPTION, RESNET

6.1 AlexNet (2012) — The Breakthrough

Component	Detail
Architecture	5 Conv layers + 3 FC layers → 1000-class Softmax
Input	224×224 RGB images
Parameters	~60 million
Key Innovation 1	First large CNN trained on GPU (NVIDIA GTX 580)
Key Innovation 2	ReLU activation instead of sigmoid — 6× faster training
Key Innovation 3	Dropout (p=0.5) in FC layers — prevents overfitting
Key Innovation 4	Data Augmentation (random crop, horizontal flip, color jitter)
Key Innovation 5	Local Response Normalization (LRN) — less used today
Result	16.4% ImageNet top-5 error (vs 26.2% previous year)

6.2 VGGNet (2014) — Depth Matters

Component	Detail
Architecture	16–19 layers (VGG16 and VGG19)
Design Rule	ONLY 3×3 convolutions + 2×2 max pooling — uniform design
Key Insight	Two stacked 3×3 convs ≈ one 5×5 conv (same receptive field but fewer params)
Channels	Start at 64, double after each pooling: 64→128→256→512
FC Layers	Three dense layers: 4096 + 4096 + 1000-class output
Parameters	~138 million (very heavy — memory-intensive)
Result	7.3% ImageNet top-5 error

Why 3×3? It's the smallest kernel that captures left/right/up/down context. Stacking two 3×3 = 5×5 receptive field with fewer params ($2 \times 9 \times C^2$ vs $25 \times C^2$).

6.3 Inception / GoogLeNet (2014) — Multi-Scale Processing

CORE IDEA: Instead of choosing kernel size (1×1, 3×3, or 5×5), use ALL of them in parallel! Concatenate the outputs → the network learns which scale is most useful.

Inception Module: [1×1 conv] + [3×3 conv] + [5×5 conv] + [3×3 max pool] → Concatenate

1×1 Bottleneck: Before expensive 3×3/5×5 convolutions, apply 1×1 to REDUCE channels. This dramatically reduces computation while adding non-linearity.

Parameters: ~5-7M (vs VGG16's 138M!) — much more efficient

Global Average Pooling replaces large FC head → fewer params, less overfitting

6.4 ResNet (2015) — Residual Learning

The KEY insight: Instead of learning the full mapping $H(x)$, learn the RESIDUAL $F(x) = H(x) - x$. Then output = $F(x) + x$.

$$\text{Output} = F(x) + x \quad [\text{Skip/Residual Connection}]$$

Why this helps:

- Skip connections provide a 'gradient highway' — gradient can flow directly through the shortcut
- Even if $F(x) = 0$ (dead layer), output = x — identity is easy to learn
- Solves the 'degradation problem': deeper networks were performing WORSE without this
- ResNet-50 uses 'bottleneck' blocks: $1 \times 1(\text{reduce}) \rightarrow 3 \times 3 \rightarrow 1 \times 1(\text{expand})$

ResNet Variant	Layers & Notes
ResNet-18/34	Basic blocks ($3 \times 3 \rightarrow 3 \times 3 + \text{skip}$)
ResNet-50	Bottleneck blocks ($1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1 + \text{skip}$). ~25M params
ResNet-101/152	Deeper bottleneck blocks. Higher accuracy, slower inference
Result	3.6% ImageNet error — surpassed human-level (5.1%)

6.5 Architecture Comparison

Architecture	Year Params Error Key Innovation
AlexNet	2012 60M 16.4% ReLU, GPU, Dropout
VGGNet-16	2014 138M 7.3% Very deep, uniform 3×3
GoogLeNet	2014 5M 6.7% Inception module, 1×1 bottleneck
ResNet-50	2015 25M 3.6% Skip connections, residual learning
DenseNet	2017 8M <3% Dense connections (all layers connected)

6.6 Sample Questions

[Easy] What is a skip connection in ResNet and why is it important?

Answer: A skip connection (also called residual/shortcut connection) adds the layer's input directly to its output: $y = F(x) + x$. Importance: (1) Gradient flows directly backward through the shortcut — prevents vanishing gradient; (2) The layer only needs to learn the residual (difference), which is easier; (3) In the worst case, the layer can learn $F(x)=0$ and just pass identity, so adding layers never hurts; (4) Enabled training of networks with 100+ layers.

[Medium] Why does VGG use only 3×3 convolutions instead of larger kernels like 7×7 ?

Answer: Two stacked 3×3 conv layers have the same receptive field as one 5×5 , and three 3×3 layers match one 7×7 . But they use fewer parameters: $2 \times (3 \times 3 \times C^2) = 18C^2$ vs $(5 \times 5 \times C^2) = 25C^2$ — a 28% savings. Three 3×3 vs one 7×7 : $27C^2$ vs $49C^2$ — a 45% savings! Additionally, stacking provides more non-linearities (more ReLU activations), enabling more complex function learning. This is why small kernels are universally preferred.

[Hard] GoogLeNet uses Global Average Pooling instead of Fully Connected layers. Why? What are the trade-offs?

Answer: Traditional CNNs end with Flatten+FC layers (AlexNet FC has ~37M of 60M total params). GAP computes the spatial average of each feature map → 1 value per channel → directly connect to output. Trade-offs: Advantages: (1) Dramatic parameter reduction — with 512 feature maps, FC needs $512 \times 7 \times 7 = 25K$ inputs per neuron; GAP reduces to 512 features total; (2) Implicit regularization — spatial averaging acts like built-in pooling; (3) Input-size independence — works with any spatial dimensions. Disadvantages: (1) Less expressive — FC can combine features across all channels; (2) May underfit for complex tasks. Modern networks (ResNet, EfficientNet) all use GAP.

TOPIC 07: NORMALIZATION & REGULARIZATION

7.1 Batch Normalization

BatchNorm normalizes the activations of each layer across the mini-batch. This keeps activations from becoming too large or small (internal covariate shift).

$$\hat{\mathbf{x}}_i = (\mathbf{x}_i - \mu_B) / \sqrt{(\sigma^2_B + \epsilon)}$$

$$\mathbf{y} = \gamma \cdot \hat{\mathbf{x}}_i + \beta$$

Where: μ_B = batch mean, σ^2_B = batch variance, ϵ = small constant (1e-8), γ = learnable scale, β = learnable shift

Aspect	Detail
When Applied	After linear transform, before activation: $\mathbf{z} \rightarrow \text{BN} \rightarrow \text{activation}$
Training	Uses batch mean/variance
Inference	Uses running average of mean/variance collected during training
Benefits	Faster convergence, higher learning rates, regularization effect
Limitation	Needs batch size $\geq 8-16$; doesn't work well for RNNs/Transformers

7.2 Layer Normalization

LayerNorm normalizes across the FEATURE dimension for each sample independently — not across the batch. This works with batch size = 1 and variable-length sequences.

$$\hat{\mathbf{x}} = (\mathbf{x} - \mu_{\text{layer}}) / \sqrt{(\sigma^2_{\text{layer}} + \epsilon)}$$

	BatchNorm
Normalizes over	Batch dimension
Batch size sensitive	YES — needs large batches
Works with RNNs/Transformers	Poorly
Training/Inference difference	YES — different statistics

7.3 Dropout

Dropout randomly sets neuron outputs to zero during training with probability p (typically 0.5). This prevents co-adaptation — neurons can't rely on specific other neurons.

INVERTED DROPOUT (used in PyTorch): During training, surviving neurons are scaled by $1/(1-p)$ to maintain expected activation magnitude. During inference, no dropout is applied.

`nn.Dropout(p=0.5)` — in training mode: 50% neurons zeroed, survivors $\times 2$
 — in eval mode: no dropout (`model.eval()` disables it)

NOTE: Always call `model.eval()` during evaluation! Forgetting causes different behavior.

7.4 L1 and L2 Regularization

L2: $\text{Loss_total} = \text{Loss_original} + \lambda \sum w_i^2$ (Weight Decay)

L1: $\text{Loss_total} = \text{Loss_original} + \lambda \sum |w_i|$ (Lasso — encourages sparsity)

In PyTorch: optimizer = torch.optim.SGD(model.parameters(), lr=0.01, weight_decay=1e-4)

7.5 Sample Questions

[Easy] What is the difference between Batch Normalization and Layer Normalization?

Answer: BatchNorm normalizes across the batch dimension (computes mean/variance per feature across all samples in the batch). LayerNorm normalizes across the feature dimension (computes mean/variance per sample across all features). Key practical differences: BatchNorm requires large batches (noisy with small batches), while LayerNorm is batch-size independent. BatchNorm behaves differently at training vs inference (uses running stats at inference). LayerNorm is identical at training and inference. LayerNorm is standard in Transformers/NLP; BatchNorm is standard in CNNs.

[Medium] Why does Dropout help with generalization? What is the ensemble interpretation?

Answer: Dropout forces the network to learn redundant representations — since any neuron can be dropped, the network can't rely on any single neuron. Each training step uses a different subset of neurons (a different 'thinned network'). The ensemble interpretation: with n neurons, dropout trains 2^n different sub-networks simultaneously (by weight sharing). At inference, using all neurons with scaling approximates averaging over all these sub-networks. This averaging tends to reduce variance without increasing bias — the hallmark of ensemble methods.

[Hard] Why does BatchNorm have two learnable parameters (γ and β)? What would happen if we only normalized without them?

Answer: After normalization, all activations have mean=0, variance=1. This is restrictive — the network may need activations with different distributions for optimal representation. Without γ, β : every layer is forced to output zero-mean, unit-variance data, severely limiting what functions the network can represent. With γ (scale) and β (shift): the network CAN undo the normalization if beneficial (by learning $\gamma=\text{std}$, $\beta=\text{mean}$), while still benefiting from the stable training properties of BN during optimization. They're initialized as $\gamma=1$, $\beta=0$ (pure normalization), and the network learns the optimal values through backprop.

TOPIC 08: RECURRENT NEURAL NETWORKS (RNNs)

8.1 Why RNNs? Sequential Data

Standard networks (MLP, CNN) assume fixed-size, independent inputs. But real-world data is often sequential: text, speech, time series, video. RNNs process sequences by maintaining a 'hidden state' that carries information across time steps.

□ Intuition: Think of RNN as reading a book word by word.

At each word, you have: the current word (input x_t) + your memory of previous words (h_{t-1}).

You update your understanding → new memory state h_t .

At the end of the sentence, h_T captures the overall meaning.

8.2 Vanilla RNN

$$h_t = \tanh(W_{hh} \cdot h_{t-1} + W_{xh} \cdot x_t + b_h)$$

$$\hat{y}_t = \text{softmax}(W_{hy} \cdot h_t + b_y)$$

Where: h_t = hidden state at time t , x_t = input at time t , W_{hh} = recurrent weight, W_{xh} = input weight

8.3 Sequence Pattern Types

Type	Pattern & Example
One-to-One	Single input → single output. Standard MLP/CNN classification.
One-to-Many	Single input → sequence. Image → caption (image captioning)
Many-to-One	Sequence → single output. Text → sentiment (sentiment analysis)
Many-to-Many (synced)	Aligned seq → seq. Video classification frame by frame.
Many-to-Many (async)	Input seq → output seq. Machine translation (encoder-decoder)

8.4 Backpropagation Through Time (BPTT)

Training RNNs requires unrolling the network across all time steps and applying backpropagation. Gradients flow backward through time.

$$\partial L / \partial W_{hh} = \sum_t \partial L_t / \partial W_{hh}$$

BPTT Challenge: Gradient flows through many matrix multiplications (one per time step).

→ Long sequences → many multiplications → vanishing or exploding gradients

TRUNCATED BPTT: Only backpropagate through last k steps (not full sequence).

→ Computationally feasible but loses long-range dependencies.

GRADIENT CLIPPING: Clip gradient norm to max value (e.g., 5.0) to prevent exploding.

`torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)`

8.5 LSTM — Long Short-Term Memory

LSTM solves vanishing gradient by adding a 'cell state' (C_t) — a separate memory highway that can store information over long sequences using additive updates (not multiplicative).

Forget Gate — what to erase from memory

$$f_t = \sigma(W_f h_{t-1} + U_f x_t + b_f)$$

$f_t \in (0,1)$: $f_t \approx 0 \rightarrow$ forget; $f_t \approx 1 \rightarrow$ keep. Sigmoid keeps values in $[0,1]$.

Input Gate — what new info to add

$$i_t = \sigma(W_i h_{t-1} + U_i x_t + b_i)$$

$$\tilde{c}_t = \tanh(W_c h_{t-1} + U_c x_t + b_c) \quad [\text{candidate values}]$$

Cell State Update — the memory highway

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$\odot$ = element-wise multiplication. ADDITIVE update — gradients flow directly, no vanishing!

Output Gate — what to output

$$o_t = \sigma(W_o h_{t-1} + U_o x_t + b_o)$$

$$h_t = o_t \odot \tanh(c_t)$$

8.6 GRU — Gated Recurrent Unit

GRU is a simplified LSTM with 2 gates instead of 3. Fewer parameters, comparable performance.

$$r_t = \sigma(W_r h_{t-1} + U_r x_t + b_r) \quad [\text{Reset gate}]$$

$$z_t = \sigma(W_z h_{t-1} + U_z x_t + b_z) \quad [\text{Update gate}]$$

$$\tilde{h}_t = \tanh(W(r_t \odot h_{t-1}) + U x_t) \quad [\text{Candidate state}]$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad [\text{Final hidden state}]$$

	LSTM
Gates	3 (forget, input, output)
Cell State	Separate $c_t + h_t$
Parameters	More (~4× hidden size)
Performance	Better for very long sequences
Training Speed	Slower (more params)

8.7 Bidirectional RNNs

Standard RNNs process sequences left-to-right. BiRNNs process in BOTH directions simultaneously and concatenate the outputs.

$$h_t = [h_t^{\text{forward}} ; h_t^{\text{backward}}] \quad [\text{Concatenation}]$$

Use case: When future context helps understand current position. E.g., in NLP: 'The bank [of the river]' — 'of the river' clarifies that 'bank' means riverbank, not financial bank.

8.8 Sample Questions

[Easy] What is the vanishing gradient problem in RNNs, and why is it worse than in MLPs?

Answer: In RNNs, gradients must flow backward through time (BPTT), passing through the same weight matrix W_{hh} at each time step. Gradient at step 1 = $\partial L / \partial h_1 = \partial L / \partial h_T \times (W_{hh})^{T-1}$. If $\|W_{hh}\| < 1$, this exponentially shrinks; if $\|W_{hh}\| > 1$, it exponentially grows. For 100 time steps with 0.9: $0.9^{100} \approx 2.7 \times 10^{-5}$ — essentially zero. In MLPs, each layer has different weights, so the product of different

matrices doesn't consistently shrink. RNNs multiply the SAME matrix T times, making the problem much worse for long sequences.

[Medium] Explain how LSTM's cell state prevents the vanishing gradient problem.

Answer: The key equation is: $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$. This is an ADDITIVE update (not multiplicative chain). Gradient through cell state: $\partial c_t / \partial c_{t-1} = f_t$ (element-wise, values close to 1). Unlike vanilla RNN where $\partial h_t / \partial h_{t-1}$ involves a full matrix multiplication through tanh (squashing gradients), the cell state gradient only gets multiplied by the forget gate (a scalar close to 1). This creates a 'constant error carousel' — information can flow unchanged for many steps if the forget gate is open (≈ 1).

[Hard] During LSTM training, you observe that the model performs well on short sequences but poorly on very long ones. What might be wrong and how would you diagnose/fix it?

Answer: DIAGNOSIS: (1) Check forget gate activations — if $f_t \approx 0$ consistently, memory is being erased too quickly; (2) Check gradient norms across time steps for vanishing gradients; (3) Plot loss vs sequence length to confirm the pattern. POSSIBLE CAUSES & FIXES: (1) Forget gate initialized to 0 → solution: initialize forget gate bias to 1.0 (Jozefowicz et al., 2015) so initially $f_t \approx \sigma(1) \approx 0.73$; (2) Learning rate too high → gradient explosion → use gradient clipping; (3) Sequence too long for truncated BPTT window → increase k ; (4) Architecture: use stacked LSTMs (2-3 layers) for more representational power; (5) Attention mechanism: instead of relying on single final hidden state for encoder-decoder, use attention over all encoder states.

TOPIC 09: AUTOENCODERS & IMAGE SEGMENTATION

9.1 Autoencoders

An autoencoder learns a compact (lower-dimensional) representation of data in an unsupervised manner. It has two parts: Encoder (compresses input) and Decoder (reconstructs from compressed form).

Architecture: Input $X \rightarrow$ Encoder $\rightarrow$ Latent Code $z \rightarrow$ Decoder $\rightarrow$ Reconstruction $\hat{X}$

Loss: Reconstruction Error (MSE for continuous data)

$$L = (1/n) \sum ||x_i - \hat{x}_i||^2$$

For binary data: Binary Cross-Entropy

$$L = -\sum [x_i \log(\hat{x}_i) + (1-x_i) \log(1-\hat{x}_i)]$$

Latent code z is forced to be small $\rightarrow$ network must learn efficient representation

9.2 Autoencoder Variants

Variant	Key Feature & Use Case
Standard AE	Minimize reconstruction error. Use: Dimensionality reduction, feature learning.
Sparse AE	Add sparsity constraint: only few neurons active. Encourages distinct features.
Denoising AE	Train to reconstruct CLEAN input from NOISY input. Learns robust features.
Contractive AE	Penalize large Jacobian: $ J_f ^2_F$ added to loss. Smooth latent space.
Variational AE (VAE)	Latent space is a probability distribution (μ, σ) . Allows sampling & generation.

9.3 Image Segmentation Overview

Method	Architecture & Key Feature
FCN (Fully Conv Net)	Converts FC layers to conv. End-to-end pixel prediction. Upsamples with transpose conv.
SegNet	Encoder-decoder using max pooling INDICES from encoder for precise upsampling.
U-Net	Encoder-decoder with SKIP CONNECTIONS concatenating encoder features to decoder.
PSPNet	Pyramid Pooling Module: multiple pooling scales (1×1, 2×2, 4×4) for multi-scale context.
DeepLab	Atrous convolution + ASPP + CRF post-processing for boundary refinement.

9.4 U-Net Architecture

U-Net was designed for biomedical segmentation with very few labeled samples. Its key innovation: skip connections that concatenate encoder feature maps with decoder feature maps.

ENCODER (Contracting Path): Conv3×3 → ReLU → MaxPool (downsampling × 4)
64 → 128 → 256 → 512 → 1024 channels, spatial size halves each step

BOTTLENECK: 1024 channels at smallest spatial resolution

DECODER (Expanding Path): UpSampling2D → Concatenate(skip connection) → Conv3×3
Skip connections bring back fine-grained spatial details lost during downsampling

OUTPUT: 1×1 Conv → sigmoid/softmax per pixel

KEY ADVANTAGE: Skip connections restore fine details needed for precise boundaries

9.5 DeepLab & Atrous Spatial Pyramid Pooling

DeepLab uses atrous (dilated) convolution to maintain spatial resolution while increasing receptive field. The ASPP module processes with multiple dilation rates simultaneously:

ASPP: [1×1 conv] + [3×3, rate=6] + [3×3, rate=12] + [3×3, rate=18] + [GAP] →
Concatenate → 1×1 conv

CRF (Conditional Random Field) post-processing further refines boundaries:

$$E(\mathbf{x}, \mathbf{y}) = \sum_i \varphi(\mathbf{x}_i, \mathbf{y}_i) + \sum_{i,j} \psi(\mathbf{x}_i, \mathbf{x}_j)$$

Unary term φ : CNN prediction per pixel. Pairwise term ψ : encourages spatially close, similar-color pixels to have the same label.

9.6 Segmentation Loss Functions

Dice Loss

$$\text{Dice}(P, G) = (2\sum p_i g_i + \epsilon) / (\sum p_i + \sum g_i + \epsilon)$$

$$L_{\text{Dice}} = 1 - \text{Dice}(P, G)$$

Best for class imbalance (when background $\gg$ foreground, e.g., tumor segmentation).

Focal Loss

$$L_{\text{Focal}} = -\alpha(1-p_t)^\gamma \log(p_t)$$

$\gamma=2$ focuses on hard examples; easy (well-classified) examples get down-weighted. Used in RetinaNet.

9.7 Sample Questions

[Easy] What is the role of skip connections in U-Net?

Answer: During encoding (downsampling), spatial resolution decreases — fine-grained details (exact edges, textures) are lost. The decoder needs to produce pixel-accurate segmentation maps, which requires this spatial detail. Skip connections directly concatenate the encoder's feature maps (before pooling) to the corresponding decoder feature maps. This provides: (1) High-resolution spatial information for precise localization; (2) Both semantic context (from bottleneck) and spatial detail (from skip connections); (3) Better gradient flow during training. Without skip connections, U-Net would only have coarse, blurry segmentation boundaries.

[Medium] Compare Dice Loss and Cross-Entropy Loss for image segmentation. When should you use each?

Answer: Cross-Entropy: Computes loss per pixel independently → with class imbalance (tiny tumor in large background), the model learns to predict 'background' everywhere and still gets low CE loss. Dice Loss: Computes overlap between prediction and ground truth → directly optimizes the segmentation metric (Dice/F1), forcing the model to find the actual object even if it's small. Use Dice Loss (or Dice + CE combined) when: extreme class imbalance (medical segmentation), small objects dominate; Use CE when: classes are relatively balanced, binary per-pixel decision.

[Hard] Explain how DeepLab's ASPP addresses the multi-scale problem in segmentation.

Answer: Problem: Objects appear at multiple scales (tiny birds, large elephants), and stride/pooling reduces resolution losing fine detail. ASPP (Atrous Spatial Pyramid Pooling) applies the SAME feature map to 4-5 parallel branches simultaneously: (1) 1×1 conv: captures local context (1×1 effective receptive field); (2) 3×3 dilated rate=6: 13×13 effective field; (3) 3×3 dilated rate=12: 25×25 effective field; (4) 3×3 dilated rate=18: 37×37 effective field; (5) Global Average Pooling: captures global context. All outputs concatenated + 1×1 conv to fuse. Result: The network can simultaneously reason about small details (rate=1) and large context (rate=18), making it robust to objects of any size.

TOPIC 10: PYTORCH CODE PATTERNS & NOTEBOOK INSIGHTS

10.1 Model Definition Pattern

```
class MyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 64, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(64)
        self.relu = nn.ReLU()
        self.pool = nn.MaxPool2d(2, 2)
        self.fc = nn.Linear(64*16*16, 10)

    def forward(self, x):
        x = self.pool(self.relu(self.bn1(self.conv1(x))))
        x = x.flatten(start_dim=1) # or x.view(x.size(0), -1)
        x = self.fc(x)
        return x
```

10.2 Transfer Learning — Two-Phase Training (from defect_detection.ipynb)

```
# Phase 1: Freeze backbone, train only classifier head
model = MobileNetV2(pretrained=True)
for param in model.parameters():
    param.requires_grad = False # Freeze ALL backbone layers
model.classifier = nn.Linear(1280, num_classes) # New trainable head
# Train for ~9 epochs with high lr (e.g., 0.01)

# Phase 2: Fine-tune entire network (unfreeze)
for param in model.parameters():
    param.requires_grad = True
optimizer = Adam(model.parameters(), lr=0.0001) # MUCH lower lr!
# Train for ~11 more epochs

WHY TWO PHASES? Phase 1 lets the new head learn without corrupting
pre-trained weights. Phase 2 allows fine-tuning with stable starting point.
```

10.3 CNN Dimension Tracking (CRITICAL for Exams)

Layer	Formula for Output Size
Conv2d	$H_{out} = \text{floor}((H_{in} + 2P - K) / S) + 1$
MaxPool2d	$H_{out} = \text{floor}((H_{in} - K) / S) + 1$
Flatten	$\text{Size} = C \times H \times W$
Linear	Shape: (batch, out_features)
Dilated Conv	Effective $K = (K-1) \times \text{dilation} + 1$, then apply standard formula

Step-by-step example (from MidSem practice):

Input (4, 3, 32, 32) → **Conv**(3→16, k=5, s=2, p=1) → (4, 16, 16, 16)
 → **Conv**(16→32, k=3, s=1, p=0) → (4, 32, 14, 14) → **MaxPool**(k=2,s=2) → (4, 32, 7, 7)

10.4 Key PyTorch Functions Reference

Function	Purpose
nn.Conv2d(C _{in} , C _{out} , k, stride, padding)	2D convolution layer
nn.BatchNorm2d(C)	Batch normalization for conv output
nn.MaxPool2d(k, stride)	Max pooling — no learnable params
nn.Dropout(p=0.5)	Randomly zero p fraction of features
nn.Linear(in, out)	Fully connected layer
loss.backward()	Compute gradients (backprop)
optimizer.zero_grad()	Clear accumulated gradients
optimizer.step()	Update weights using computed gradients
model.eval()	Switch to eval mode (affects BN, Dropout)
model.train()	Switch to training mode
torch.no_grad()	Context manager: no gradient computation
x.flatten(start_dim=1)	Flatten spatial dims for FC layer input

10.5 Image Captioning Architecture (from epgd_Image_captioning.ipynb)

ENCODER: Pre-trained CNN (ResNet) extracts image features → 2048-dim vector
 Image → ResNet → Global Avg Pool → Linear(2048, embed_dim) → h₀ for LSTM

DECODER: LSTM generates caption word by word
 At each step: [word embedding] + [image context] → LSTM → next word

TRAINING: Teacher forcing — use ground truth word as next input (not model's prediction)
 Makes training stable and fast

INFERENCE: Greedy decoding — always pick the most probable next word
 Or: Beam search — keep top-k candidates at each step (better quality)

VOCABULARY: Special tokens: <pad>=0, <start>=1, <end>=2, <unknown>=3

10.6 Segmentation Evaluation Metrics

Pixel Accuracy = (correctly classified pixels) / (total pixels)

IoU (per class) = TP / (TP + FP + FN)

Mean IoU = (1/C) Σ IoU_c [averaged over C classes]

IoU (Intersection over Union) is the standard segmentation metric. It measures overlap between predicted and ground-truth masks. IoU=1 means perfect overlap; IoU=0 means no overlap.

TOPIC FS: MASTER FORMULA SHEET

□ All critical formulas in one place — perfect for last-day revision!

Activation Functions

Name	Formula
ReLU	$f(x) = \max(0, x)$ $f'(x) = 1$ if $x > 0$ else 0
Sigmoid	$\sigma(x) = 1/(1+e^{-x})$ $\sigma'(x) = \sigma(x)(1-\sigma(x)) \leq 0.25$
Tanh	$\tanh(x) = (e^x - e^{-x})/(e^x + e^{-x})$ range: $(-1, 1)$
Softmax	$\sigma(y_i) = e^{y_i} / \sum_j e^{y_j}$ (converts logits to probabilities)
Swish	$f(x) = x \cdot \sigma(x)$ (smooth, non-monotonic)
Leaky ReLU	$f(x) = \max(\alpha x, x)$ where $\alpha \approx 0.01$

Loss Functions

Loss	Formula & When to Use
MSE	$L = (1/n) \sum (y - \hat{y})^2$ Regression tasks
Binary CE	$L = -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$ Binary classification, reconstruction
Categorical CE	$L = -\sum y_i \log(\hat{y}_i)$ Multi-class classification
Dice Loss	$L = 1 - (2\sum p_i g_i + \epsilon) / (\sum p_i + \sum g_i + \epsilon)$ Segmentation (imbalanced)
Focal Loss	$L = -\alpha(1-p_t)^\gamma \log(p_t)$ Hard example mining

CNN Geometry

```

Output_size = floor( (Input - Kernel + 2×Padding) / Stride ) + 1
Effective Kernel (Dilated) = (Kernel - 1) × Dilation + 1
Parameters (Conv) = Kernel_H × Kernel_W × C_in × C_out + C_out
Parameters (Linear) = in_features × out_features + out_features

```

Normalization

```

BatchNorm:  $\hat{x} = (x - \mu_B) / \sqrt{(\sigma_B^2 + \epsilon)}$    ;    $y = \gamma \hat{x} + \beta$ 
LayerNorm:  $\hat{x} = (x - \mu_L) / \sqrt{(\sigma_L^2 + \epsilon)}$    ;    $y = \gamma \hat{x} + \beta$ 

```

Optimization

```

SGD:  $w_{t+1} = w_t - \alpha \partial L / \partial w$ 
Momentum:  $v_t = \beta v_{t-1} + (1-\beta)g_t$    ;    $w = w - \alpha \cdot v_t$ 
Adam:  $m_t = \beta_1 m_{t-1} + (1-\beta_1)g_t$    ;    $v_t = \beta_2 v_{t-1} + (1-\beta_2)g_t^2$ 

```

$$\hat{m} = m_t / (1 - \beta_1^t) \quad ; \quad \hat{v} = v_t / (1 - \beta_2^t) \quad ; \quad w = w - \alpha \cdot \hat{m} / (\sqrt{\hat{v}} + \epsilon)$$

RNN / LSTM / GRU

RNN: $h_t = \tanh(W_{hh} \cdot h_{t-1} + W_{xh} \cdot x_t + b)$
 LSTM Forget: $f_t = \sigma(W_f h_{t-1} + U_f x_t + b_f)$
 LSTM Input: $i_t = \sigma(W_i h_{t-1} + U_i x_t + b_i) \quad ; \quad \tilde{c}_t = \tanh(W_c h_{t-1} + U_c x_t + b_c)$
 LSTM Cell: $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$
 LSTM Output: $o_t = \sigma(W_o h_{t-1} + U_o x_t + b_o) \quad ; \quad h_t = o_t \odot \tanh(c_t)$
 GRU Reset: $r_t = \sigma(W_r h_{t-1} + U_r x_t) \quad ; \quad \text{Update: } z_t = \sigma(W_z h_{t-1} + U_z x_t)$
 GRU Hidden: $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tanh(W(r_t \odot h_{t-1}) + Ux_t)$

Segmentation Metrics

$$\text{IoU} = TP / (TP + FP + FN)$$

$$\text{Dice} = 2 \times TP / (2 \times TP + FP + FN) = 2 \times \text{IoU} / (1 + \text{IoU})$$

 Pixel Accuracy = $\sum_i n_{ii} / \sum_i t_i$ where n_{ii} = correctly classified pixels of class i

$$\text{Mean IoU} = (1/C) \sum_i n_{ii} / (t_i + \sum_j n_{ji} - n_{ii})$$

TOPIC QR: QUICK REVISION — 1-PAGE SUMMARIES PER TOPIC

⚡ NEURAL NETWORKS — QUICK REVISION

- **Perceptron:** $z = w^T x + b$, output = $f(z)$
- MLP needs non-linear activations to learn non-linear functions
- Universal Approximation: 1 hidden layer + enough neurons = any function
- Deep > Wide: $O(m^L)$ linear regions vs $O(m)$
- XOR requires hidden layer — single perceptron cannot solve it
- Backprop = chain rule applied backward through computation graph
- Vanishing gradient: Sigmoid derivative max 0.25; deep chains → near-zero gradient
- Solution: ReLU (gradient=1 when active), Skip connections, BatchNorm
- Dying ReLU: neuron stuck at 0 forever → Use Leaky ReLU or He initialization

⚡ CNNs — QUICK REVISION

- **Convolution:** filter slides across image → feature map
- **Output size = $\text{floor}((I_n - K + 2P) / S) + 1$ ← MEMORIZE THIS**
- Each filter detects ONE feature type; learnable during training
- Multi-channel: filter depth = input depth; C_{out} filters → C_{out} output maps
- Max Pooling: no params, provides translation invariance, reduces size
- Dilation: larger receptive field WITHOUT extra params or resolution loss
- AlexNet: ReLU + GPU + Dropout. VGG: uniform 3×3. ResNet: Skip connections.
- BatchNorm: normalizes per batch, reduces covariate shift, acts as regularizer
- Dropout: randomly zeros neurons during training, disabled at inference

⚡ RNNs & LSTMs — QUICK REVISION

- **RNN:** $h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t)$ — same weights reused at each step
- Vanishing gradient in RNN: multiplying W_{hh} T times → exponential decay/growth
- LSTM: 3 gates (forget/input/output) + cell state c_t — additive updates prevent vanishing
- Forget gate: what to erase (close to 0). Input gate: what to write.
- Cell state: $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ — gradient highway!
- GRU: 2 gates (reset/update), fewer params than LSTM, similar performance
- Bidirectional: process forward + backward, concatenate hidden states
- BPTT: backprop through time. Truncated BPTT for long sequences.
- Gradient clipping: clip norm to prevent exploding gradients

TOPIC PQ: PREDICTED EXAM QUESTIONS WITH FULL SOLUTIONS

- Based on past quiz/exam patterns and course content — high probability exam questions!

Section A: MCQ-Style Predicted Questions

Q1. What is the effective receptive field of a 3×3 dilated convolution with dilation=2?

Answer: 5×5

Calculation: Effective size = $(K-1) \times d + 1 = (3-1) \times 2 + 1 = 5$

Note: The 3×3 kernel now 'sees' a 5×5 region but still has only 9 learnable parameters.

Q2. In a CNN, which of these has learnable parameters?

a) MaxPool2d b) ReLU c) BatchNorm2d d) Dropout

Answer: c) BatchNorm2d

BatchNorm has learnable scale (γ) and shift (β) parameters.

MaxPool, ReLU, Dropout have NO learnable parameters.

Q3. Which optimizer is MOST recommended for deep learning tasks?

Answer: Adam (with $\text{lr}=0.001$, $\beta_1=0.9$, $\beta_2=0.999$)

Adam combines momentum (fast convergence) with adaptive learning rates (robust to sparse gradients).

Works well across most deep learning tasks without extensive hyperparameter tuning.

Q4. What is the output of `model.eval()` that changes in a model using Dropout and BatchNorm?

Answer: Both Dropout and BatchNorm behavior changes:

- Dropout: DISABLED (all neurons active, no zeroing)

- BatchNorm: Uses RUNNING MEAN/VARIANCE (collected during training) instead of batch statistics

CRITICAL: Always call `model.eval()` before evaluation and `model.train()` before training!

Section B: Long Answer Questions

Q5. Design a CNN for binary classification of 64×64 grayscale images. Calculate output dimensions and total parameters at each layer.

Model Design:

Conv2d(1, 32, k=3, s=1, p=1) + ReLU + MaxPool(2,2)

Conv2d(32, 64, k=3, s=1, p=1) + ReLU + MaxPool(2,2)

Flatten + Linear(64×16×16, 128) + ReLU + Linear(128, 1) + Sigmoid

Dimension Trace:

Input: (B, 1, 64, 64)

After Conv1+Pool: $H = \text{floor}((64+2-3)/1)+1 = 64$; MaxPool $\rightarrow 32$. Shape: (B, 32, 32, 32)

After Conv2+Pool: $H = 32$; MaxPool $\rightarrow 16$. Shape: (B, 64, 16, 16)

Flatten: $64 \times 16 \times 16 = 16384$

After Linear: (B, 128) $\rightarrow$ (B, 1)

Parameter Count:

Conv1: $3 \times 3 \times 1 \times 32 + 32 = 288 + 32 = 320$

Conv2: $3 \times 3 \times 32 \times 64 + 64 = 18,432 + 64 = 18,496$

Linear1: $16384 \times 128 + 128 = 2,097,152 + 128 = 2,097,280$

Linear2: $128 \times 1 + 1 = 129$

TOTAL: $320 + 18,496 + 2,097,280 + 129 = 2,116,225$ parameters

Note: Most parameters are in the Linear layer — use GAP to reduce this!

Q6. Trace through LSTM gates for a single time step. Given: $h_{t-1} = [0.5, -0.3]$, $x_t = [1.0, 0.5]$, and simplify with all weights=0.5, biases=0.

LSTM Step-by-Step Trace (simplified — treat as single values for clarity):

Step 1 - Forget Gate:

$$\begin{aligned} f_t &= \sigma(W_f \cdot h_{t-1} + U_f \cdot x_t) = \sigma(0.5 \times 0.5 + 0.5 \times (-0.3) + 0.5 \times 1.0 + 0.5 \times 0.5) \\ &= \sigma(0.25 - 0.15 + 0.5 + 0.25) = \sigma(0.85) \approx 0.70 \\ &\rightarrow 70\% \text{ of previous cell state is retained} \end{aligned}$$

Step 2 - Input Gate: $i_t = \sigma(\text{same computation}) \approx 0.70$

Step 3 - Candidate: $\tilde{c}_t = \tanh(\text{same computation}) \approx \tanh(0.85) \approx 0.69$

Step 4 - Cell State Update:

Assume $c_{t-1} = 0.2$

$$c_t = f_t \times c_{t-1} + i_t \times \tilde{c}_t = 0.70 \times 0.2 + 0.70 \times 0.69 = 0.14 + 0.483 = 0.623$$

Step 5 - Output Gate: $o_t = \sigma(0.85) \approx 0.70$

Step 6 - Hidden State: $h_t = o_t \times \tanh(c_t) = 0.70 \times \tanh(0.623) = 0.70 \times 0.553 \approx 0.387$

Key insight: Cell state grew from 0.2 to 0.623 — memory was UPDATED, not erased!

Q7. Compare U-Net and SegNet for image segmentation. When would you choose each?

U-Net:

Architecture: Encoder-Decoder + Skip connections (feature concatenation)

Memory: Higher (stores full feature maps for skip connections)

Accuracy: Superior boundary detection (skip connections restore fine details)

Parameters: Moderate

Best for: Medical imaging, small datasets, tasks requiring precise boundaries

SegNet:

Architecture: Encoder-Decoder + Max pooling INDICES (not features)

Memory: Lower (only stores indices, not full feature maps)

Accuracy: Good boundary preservation, efficient
 Parameters: Similar to U-Net
 Best for: Resource-constrained environments (mobile/edge), road scene segmentation

Choose U-Net when: Precision is critical (medical), you have GPU memory
 Choose SegNet when: Memory is constrained, efficiency is priority

Section C: Code Output Questions

Q8. What does this code output? (from MidSem practice)

```
x = torch.tensor(3.0, requires_grad=True)
y = torch.tensor(4.0, requires_grad=True)
z = x**2 + 2*x*y + y**3
z.backward()
print('x.grad:', x.grad) → ?
print('y.grad:', y.grad) → ?
```

SOLUTION:

$$z = x^2 + 2xy + y^3$$

$$\partial z / \partial x = 2x + 2y = 2(3) + 2(4) = 6 + 8 = 14$$

$$\partial z / \partial y = 2x + 3y^2 = 2(3) + 3(16) = 6 + 48 = 54$$

OUTPUT: x.grad: tensor(14.) | y.grad: tensor(54.)

Q9. CustomCNN receives (4, 3, 64, 64) input. What is the output shape? Does it crash?

Architecture: Conv(3→16, k=5, s=2, p=1) → Conv(16→32, k=3, s=1, p=0) → MaxPool(2,2)
 → Flatten → Linear(32×6×6, 10) [Linear expects 1152 inputs]

Tracing (64×64 input):

After Conv1: $\text{floor}((64+2-5)/2)+1 = \text{floor}(61/2)+1 = 30+1 = 31$. Shape: (4,16,31,31)

After Conv2: $\text{floor}((31-3)/1)+1 = 29$. Shape: (4,32,29,29)

After MaxPool: $\text{floor}((29-2)/2)+1 = 14$. Shape: (4,32,14,14)

Flatten: $32 \times 14 \times 14 = 6272$ features

Linear expects: $32 \times 6 \times 6 = 1152$ features

→ MISMATCH! Code CRASHES at self.classifier(x)

Error: mat1 and mat2 shapes cannot be multiplied (4×6272 and 1152×10)

FIX: Use AdaptiveAvgPool2d(output_size=(6,6)) before flatten to ensure fixed size.

Section D: Common Exam Traps — Be Careful!

1. model.eval() vs model.train(): ALWAYS switch modes! Eval disables Dropout, uses running BN stats.
2. optimizer.zero_grad(): MUST call before loss.backward(). Forgetting causes gradient accumulation!
3. Dilation vs Stride: Both increase receptive field but dilation PRESERVES resolution; stride REDUCES it.

4. Dice Coefficient vs Dice Loss: Dice Coefficient (higher=better); Dice Loss = $1 - \text{Dice}$ (lower=better).
5. Forget Gate bias: Should be initialized to 1.0 for LSTMs to ensure memory retention at the start.
6. Padding='same': Means output $H=W=\text{input } H=W$ ONLY for stride=1. Different behavior with stride>1.
7. C_out filters: Total parameters in Conv2d = $k \times k \times C_{\text{in}} \times C_{\text{out}} + C_{\text{out}}$ (one bias per output channel).
8. Bidirectional RNN doubles the hidden size: output dim = $2 \times \text{hidden_dim}$.
9. Softmax vs Sigmoid: Softmax for multi-class (probabilities sum to 1), Sigmoid for binary per class.
10. BatchNorm has learnable params (γ, β); LayerNorm also has learnable params; MaxPool has NONE.

TOPIC CM: COMMON MISTAKES & HOW TO AVOID THEM

Common Mistake	Correct Understanding & Tip
Confusing dilation with stride	Dilation expands receptive field WITHOUT changing output size (with same padding). Stride > 1 REDUCES output size. Both increase how much of the input one neuron 'sees'.
Thinking UAT means deep learning always works	Universal Approximation Theorem only guarantees a network CAN represent the function. It doesn't mean training WILL find it, or that it will GENERALIZE to new data.
Forgetting to flatten before Linear layers	Linear layers expect 1D input (per sample). Always flatten after convolutional blocks: <code>x = x.flatten(start_dim=1)</code> or <code>x.view(x.size(0), -1)</code>
Using BCE for multi-class problems	BCE (Binary Cross-Entropy) is for binary or multi-LABEL classification. For mutually exclusive classes (multi-class), use Categorical Cross-Entropy with Softmax output.
Wrong gate understanding in LSTM	Forget gate: values close to 0 = FORGET (not remember!). Close to 1 = KEEP. Input gate close to 1 = WRITE new info. Output gate close to 1 = OUTPUT info.
Not accounting for BatchNorm in eval mode	BN during training uses batch mean/variance. During eval, it uses RUNNING AVERAGES. Different behavior can cause model to perform worse at inference if <code>model.eval()</code> is not called.
Parameter count of Conv with bias	Don't forget biases! Params = $k \times k \times C_{in} \times C_{out} + C_{out}$ (one bias per output channel). Students often forget the $+C_{out}$.
Treating IoU and Dice as unrelated	Dice = $2 \times \text{IoU} / (1 + \text{IoU})$. They measure the same thing differently. Dice is equivalent to F1 score. High IoU $\leftrightarrow$ High Dice.
Thinking GRU is always better than LSTM	Not true. LSTM generally performs better on very long sequences due to separate cell state (better memory). GRU is faster to train. For most tasks: comparable performance.
Applying Dropout at inference time	Dropout MUST be disabled at inference. In PyTorch: always call <code>model.eval()</code> before <code>model(test_input)</code> . Forgetting causes random, non-deterministic predictions.

